

1. Objective

To build a user-friendly web interface for interacting with a trained deep learning prediction model, using Streamlit, so that a person with no coding experience can try a handwritten-digit classifier directly through a browser - and to close the gap between clean training data and messy real-world input (freehand drawings, photos) that a naive preprocessing pipeline cannot fully bridge.

2. Approach

1. **Verify correctness before training anything** - `train_model.py` runs a numerical gradient check (comparing hand-derived analytic gradients against a finite-difference approximation) and aborts if the two disagree, catching backpropagation bugs before wasting any training time.
2. **Train on realistic, heavily augmented data** - random rotation, independent-axis scaling (including deliberately extreme stretches/squishes), translation, elastic warping, stroke thickening/thinning, and noise, generated via the same `digit_utils.py` functions the app uses at inference time.
3. **Design a Streamlit UI** - a sidebar with model info/fingerprint and three navigable modes in the main area.
4. **Create user input forms and widgets** - a filterable sample gallery, a real drawing canvas with a brush-size slider (falling back to a click-to-paint pixel grid if the canvas component is unavailable), and a file uploader with an invert-colors toggle.
5. **Connect the UI to model prediction logic** - every mode's resulting image is run through `digit_utils.smart_preprocess_to_8x8` and then `predict_with_tta`, which averages the model's prediction over several small, realistic perturbations of the same input rather than relying on one single forward pass.
6. **Display prediction outputs** - the predicted digit, a color-coded confidence level (green/orange/red), the top-3 guesses with medal icons, and the full 10-digit probability chart.
7. **Improve usability using layout components** - a hero banner, custom CSS, a sidebar, columns, multiple expanders, metrics, and a confidence-key legend keep the interface organized despite covering three quite different workflows.

3. Source Code - digit_utils.py

Shared preprocessing used by both the app and the training script, so training and inference always agree on how an image is read:

```
"""
digit_utils.py
=====
Shared, framework-free image preprocessing used by BOTH the Streamlit app
(streamlit_app.py) and the training script (train_model.py).

Why this file exists: a model is only as good as the match between what it
was trained on and what it sees at inference time. Earlier versions of this
project trained on pristine, pre-normalized 8x8 digits but then fed the
model raw, off-center, arbitrarily-scaled freehand drawings at inference
time - a serious train/inference mismatch that no amount of clever
inference-time cropping can fully fix.

By putting the exact same crop -> scale -> center pipeline here and
importing it from both places, the training script can generate augmented
training examples that have gone through IDENTICAL preprocessing to what
the deployed app applies to a real drawing. That is the single biggest
lever for real-world accuracy.
"""

import numpy as np
from PIL import Image

def pixels_to_display_image(pixels_8x8, size=192):
    """8x8 array in [0,16] -> a crisp, upscaled PIL image for display."""
    arr = np.clip(pixels_8x8, 0, 16) / 16.0 * 255
    return Image.fromarray(arr.astype(np.uint8)).resize((size, size), Image.NEAREST)

def crop_to_ink(gray_arr, pad_frac=0.07):
    """Tightly crop a grayscale array (ink = brighter than background) down
    to just the bounding box of the drawn/written content, with a small
    margin. Returns a 0-255 float array, or None if nothing was found."""
    arr = gray_arr.astype(np.float64)
    if arr.max() <= arr.min():
        return None
    norm = (arr - arr.min()) / (arr.max() - arr.min()) * 255.0
    threshold = max(25.0, norm.max() * 0.18)
    mask = norm > threshold
    if not mask.any():
        return None
    ys, xs = np.where(mask)
    y0, y1, x0, x1 = ys.min(), ys.max(), xs.min(), xs.max()
    h, w = (y1 - y0 + 1), (x1 - x0 + 1)
    pad_y, pad_x = max(1, int(h * pad_frac)), max(1, int(w * pad_frac))
    y0, y1 = max(0, y0 - pad_y), min(norm.shape[0] - 1, y1 + pad_y)
    x0, x1 = max(0, x0 - pad_x), min(norm.shape[1] - 1, x1 + pad_x)
    return norm[y0:y1 + 1, x0:x1 + 1]

def shift_2d(arr, sy, sx):
    """Shift a 2D array by (sy, sx) pixels with zero-fill (no wraparound)."""
    out = np.zeros_like(arr)
    h, w = arr.shape
    sy0, sy1 = max(0, sy), min(h, h + sy)
    sx0, sx1 = max(0, sx), min(w, w + sx)
    oy0, oy1 = max(0, -sy), min(h, h - sy)
    ox0, ox1 = max(0, -sx), min(w, w - sx)
    out[sy0:sy1, sx0:sx1] = arr[oy0:oy1, ox0:ox1]
    return out

def center_by_mass(canvas8):
    """Nudge an 8x8 array so its center of mass sits at the middle of the
    frame - the same normalization classic digit-recognition pipelines use."""
    total = canvas8.sum()
    if total <= 0:
        return canvas8
    yy, xx = np.mgrid[0:8, 0:8]
    cy, cx = (yy * canvas8).sum() / total, (xx * canvas8).sum() / total
    sy, sx = int(round(3.5 - cy)), int(round(3.5 - cx))
    return shift_2d(canvas8, sy, sx)

def dilate3(arr):
    """3x3 max filter - thickens strokes by ~1px."""
    stack = [shift_2d(arr, dy, dx) for dy in (-1, 0, 1) for dx in (-1, 0, 1)]
    return np.maximum.reduce(stack)

def erode3(arr):
    """3x3 min filter - thins strokes by ~1px."""
    stack = [shift_2d(arr, dy, dx) for dy in (-1, 0, 1) for dx in (-1, 0, 1)]
    return np.minimum.reduce(stack)
```

```
def smart_preprocess_to_8x8(gray_arr):
    """Turn an arbitrarily-sized grayscale array (ink = brighter than
    background) into the 8x8, 0-16 format the model expects - cropped to
    the ink's bounding box, scaled to fill most of the frame, and centered
    by mass.

    Each axis is scaled independently within a min/max footprint (not
    forced to share one aspect-ratio-preserving scale factor): a strictly
    proportional resize crushes a tall, narrow shape (a slim figure-eight,
    a looping cursive digit) down to just 1-2 pixels wide, destroying the
    very detail - loops, holes, curves - that distinguishes it from a
    plain "1". Independent-axis scaling with a floor keeps thin digits
    recognizable.
    """
    cropped = crop_to_ink(gray_arr, pad_frac=0.07)
    if cropped is None:
        return np.zeros((8, 8))

    ch, cw = cropped.shape
    FRAME_MAX = 7.0 # the digit's longer side can use nearly the full frame
    FRAME_MIN = 3.2 # ...but neither side is allowed to collapse below this

    base_scale = FRAME_MAX / max(ch, cw)
    new_h = int(round(ch * base_scale))
    new_w = int(round(cw * base_scale))
    new_h = int(np.clip(new_h, FRAME_MIN, 8))
    new_w = int(np.clip(new_w, FRAME_MIN, 8))

    resized = np.array(
        Image.fromarray(cropped.astype(np.uint8)).resize((new_w, new_h), Image.LANCZOS)
    ).astype(np.float64)

    canvas = np.zeros((8, 8), dtype=np.float64)
    off_y, off_x = (8 - new_h) // 2, (8 - new_w) // 2
    canvas[off_y:off_y + new_h, off_x:off_x + new_w] = resized
    canvas = center_by_mass(canvas)

    if canvas.max() > 0:
        canvas = canvas / canvas.max() * 16.0
    return canvas

def predict_with_tta(model, pixels_8x8):
    """Run the model on several small, realistic perturbations of the same
    input - shifted by +/-1px, and with strokes slightly thickened/thinned -
    then average the resulting probabilities. Smooths out a single forward
    pass's sensitivity to exact pixel alignment and stroke weight."""
    base = np.clip(pixels_8x8, 0, 16)
    variants = [base]
    for dy in (-1, 0, 1):
        for dx in (-1, 0, 1):
            if dy == 0 and dx == 0:
                continue
            variants.append(shift_2d(base, dy, dx))
    variants.append(np.clip(dilate3(base), 0, 16))
    variants.append(np.clip(erode3(base), 0, 16))

    probs_sum = np.zeros(10, dtype=np.float64)
    for v in variants:
        x = (v / 16.0).reshape(1, 8, 8, 1).astype(np.float32)
        probs_sum += model.predict_proba(x)[0]
    return probs_sum / len(variants)
```

4. Source Code - train_model.py

Trains the CNN from scratch - forward pass, hand-derived backpropagation, and an Adam optimizer - after first verifying the gradients numerically:

```
"""
train_model.py
=====
Trains the digit-recognition CNN from scratch (forward AND backward pass,
pure NumPy - no autograd, no ML framework) and saves weights compatible
with the DigitCNN class in streamlit_app.py.

The key difference from a "normal" digits-dataset training script: every
training example is put through heavy, realistic augmentation (rotation,
anisotropic scaling, translation, stroke thickening/thinning, noise) and
then passed through digit_utils.smart_preprocess_to_8x8 - the SAME
crop/scale/center pipeline the deployed app applies to real drawings.
That closes the train/inference distribution gap directly, instead of
training on pristine textbook digits and hoping inference-time tricks
paper over the difference.

Run with: python train_model.py
```

```

Produces: cnn_digits_model.pkl (drop into the same folder as the app)
"""

import pickle
import time
import hashlib
from datetime import datetime
import numpy as np
from PIL import Image
from sklearn.datasets import load_digits

from digit_utils import smart_preprocess_to_8x8, dilate3, erode3

RNG = np.random.default_rng(42)

# -----
# Forward-pass primitives (identical math to streamlit_app.py's DigitCNN,
# but each function also returns a "cache" so we can run backprop)
# -----

def im2col(x, kh, kw, stride=1, pad=0):
    N, H, W, C = x.shape
    if pad > 0:
        x = np.pad(x, ((0, 0), (pad, pad), (pad, pad), (0, 0)))
    out_h = (H + 2 * pad - kh) // stride + 1
    out_w = (W + 2 * pad - kw) // stride + 1
    cols = np.zeros((N, out_h, out_w, kh, kw, C), dtype=x.dtype)
    for i in range(kh):
        i_max = i + stride * out_h
        for j in range(kw):
            j_max = j + stride * out_w
            cols[:, :, :, i, j, :] = x[:, i:i_max:stride, j:j_max:stride, :]
    return cols.reshape(N, out_h, out_w, kh * kw * C), out_h, out_w

def col2im(dcols, x_shape, kh, kw, stride=1, pad=0):
    """Inverse of im2col: scatter-add gradient contributions back onto the
    (unpadded) input shape."""
    N, H, W, C = x_shape
    out_h = (H + 2 * pad - kh) // stride + 1
    out_w = (W + 2 * pad - kw) // stride + 1
    dcols = dcols.reshape(N, out_h, out_w, kh, kw, C)
    Hp, Wp = H + 2 * pad, W + 2 * pad
    dx_padded = np.zeros((N, Hp, Wp, C), dtype=dcols.dtype)
    for i in range(kh):
        i_max = i + stride * out_h
        for j in range(kw):
            j_max = j + stride * out_w
            dx_padded[:, i:i_max:stride, j:j_max:stride, :] += dcols[:, :, :, i, j, :]
    if pad > 0:
        return dx_padded[:, pad:-pad, pad:-pad, :]
    return dx_padded

def conv_forward(x, W, b, stride=1, pad=1):
    k = W.shape[0]
    out_ch = W.shape[-1]
    cols, out_h, out_w = im2col(x, k, k, stride, pad)
    N = x.shape[0]
    cols_flat = cols.reshape(N * out_h * out_w, -1)
    W_col = W.reshape(-1, out_ch)
    out = (cols_flat @ W_col + b).reshape(N, out_h, out_w, out_ch)
    cache = (x, W, stride, pad, cols_flat, out_h, out_w)
    return out, cache

def conv_backward(dout, cache):
    x, W, stride, pad, cols_flat, out_h, out_w = cache
    N = x.shape[0]
    out_ch = W.shape[-1]
    k = W.shape[0]
    dout_flat = dout.reshape(N * out_h * out_w, out_ch)
    dW = (cols_flat.T @ dout_flat).reshape(W.shape)
    db = dout_flat.sum(axis=0)
    W_col = W.reshape(-1, out_ch)
    dcols = (dout_flat @ W_col.T).reshape(N, out_h, out_w, -1)
    dx = col2im(dcols, x.shape, k, k, stride, pad)
    return dx, dW, db

def relu_forward(x):
    return np.maximum(0, x), x

def relu_backward(dout, cache):
    return dout * (cache > 0)

def maxpool_forward(x, size=2, stride=2):
    N, H, W, C = x.shape
    out_h, out_w = H // stride, W // stride
    x_crop = x[:, :out_h * stride, :out_w * stride, :]

```

```

out = np.zeros((N, out_h, out_w, C), dtype=x.dtype)
mask = np.zeros_like(x_crop, dtype=bool)
for i in range(out_h):
    for j in range(out_w):
        window = x_crop[:, i * stride:i * stride + size, j * stride:j * stride + size, :]
        m = window.max(axis=(1, 2), keepdims=True)
        out[:, i, j, :] = m[:, 0, 0, :]
        mask[:, i * stride:i * stride + size, j * stride:j * stride + size, :] |= (window == m)
cache = (x.shape, mask, size, stride, out_h, out_w)
return out, cache

def maxpool_backward(dout, cache):
    x_shape, mask, size, stride, out_h, out_w = cache
    N, H, W, C = x_shape
    dx_crop = np.zeros((N, out_h * stride, out_w * stride, C))
    for i in range(out_h):
        for j in range(out_w):
            window_mask = mask[:, i * stride:i * stride + size, j * stride:j * stride + size, :]
            count = window_mask.sum(axis=(1, 2), keepdims=True)
            count = np.where(count == 0, 1, count)
            contribution = dout[:, i:i + 1, j:j + 1, :] / count
            dx_crop[:, i * stride:i * stride + size, j * stride:j * stride + size, :] += window_mask * contribution
    dx = np.zeros((N, H, W, C))
    dx[:, :out_h * stride, :out_w * stride, :] = dx_crop
    return dx

def dense_forward(x, W, b):
    return x @ W + b, (x, W)

def dense_backward(dout, cache):
    x, W = cache
    dx = dout @ W.T
    dW = x.T @ dout
    db = dout.sum(axis=0)
    return dx, dW, db

def softmax(logits):
    e = np.exp(logits - logits.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)

def cross_entropy_loss_and_grad(logits, labels):
    probs = softmax(logits)
    N = logits.shape[0]
    log_likelihood = -np.log(probs[np.arange(N), labels] + 1e-12)
    loss = log_likelihood.mean()
    dlogits = probs.copy()
    dlogits[np.arange(N), labels] -= 1
    dlogits /= N
    return loss, dlogits

# -----
# The network: Conv(1->C1) -> ReLU -> Pool -> Conv(C1->C2) -> ReLU -> Pool
#               -> Dense(-> H) -> ReLU -> Dense(-> 10) -> Softmax
# -----

C1, C2, HIDDEN = 10, 20, 64

def init_params():
    def he(shape, fan_in):
        return (RNG.standard_normal(shape) * np.sqrt(2.0 / fan_in)).astype(np.float64)

    return {
        "conv1_W": he((3, 3, 1, C1), 3 * 3 * 1),
        "conv1_b": np.zeros(C1),
        "conv2_W": he((3, 3, C1, C2), 3 * 3 * C1),
        "conv2_b": np.zeros(C2),
        "fc1_W": he((2 * 2 * C2, HIDDEN), 2 * 2 * C2),
        "fc1_b": np.zeros(HIDDEN),
        "fc2_W": he((HIDDEN, 10), HIDDEN),
        "fc2_b": np.zeros(10),
    }

def forward(x, p):
    out1, c1 = conv_forward(x, p["conv1_W"], p["conv1_b"])
    a1, ca1 = relu_forward(out1)
    pool1, cp1 = maxpool_forward(a1)
    out2, c2 = conv_forward(pool1, p["conv2_W"], p["conv2_b"])
    a2, ca2 = relu_forward(out2)
    pool2, cp2 = maxpool_forward(a2)
    flat = pool2.reshape(pool2.shape[0], -1)
    d1, cd1 = dense_forward(flat, p["fc1_W"], p["fc1_b"])
    a3, ca3 = relu_forward(d1)
    logits, cd2 = dense_forward(a3, p["fc2_W"], p["fc2_b"])
    cache = dict(c1=c1, ca1=ca1, cp1=cp1, c2=c2, ca2=ca2, cp2=cp2,
                 pool2_shape=pool2.shape, cd1=cd1, ca3=ca3, cd2=cd2)

```

```

return logits, cache

def backward(dlogits, cache):
    da3, dfc2W, dfc2b = dense_backward(dlogits, cache["cd2"])
    dd1 = relu_backward(da3, cache["ca3"])
    dflat, dfc1W, dfc1b = dense_backward(dd1, cache["cd1"])
    dpool2 = dflat.reshape(cache["pool2_shape"])
    da2 = maxpool_backward(dpool2, cache["cp2"])
    dout2 = relu_backward(da2, cache["ca2"])
    dpool1, dconv2W, dconv2b = conv_backward(dout2, cache["c2"])
    da1 = maxpool_backward(dpool1, cache["cp1"])
    dout1 = relu_backward(da1, cache["ca1"])
    _, dconv1W, dconv1b = conv_backward(dout1, cache["c1"])
    return {
        "conv1_W": dconv1W, "conv1_b": dconv1b,
        "conv2_W": dconv2W, "conv2_b": dconv2b,
        "fc1_W": dfc1W, "fc1_b": dfc1b,
        "fc2_W": dfc2W, "fc2_b": dfc2b,
    }

# -----
# Correctness check: numerical gradient checking on a tiny random batch.
# If this doesn't pass, the backward pass has a bug and training would
# silently produce a broken model - so we verify it BEFORE training.
# -----

def gradient_check():
    p = init_params()
    x = RNG.standard_normal((2, 8, 8, 1)) * 0.5
    labels = np.array([3, 7])

    logits, cache = forward(x, p)
    _, dlogits = cross_entropy_loss_and_grad(logits, labels)
    grads = backward(dlogits, cache)

    def loss_fn(params):
        logits, _ = forward(x, params)
        loss, _ = cross_entropy_loss_and_grad(logits, labels)
        return loss

    eps = 1e-5
    max_rel_err = 0.0
    for name in ["conv1_W", "conv1_b", "conv2_W", "conv2_b", "fc1_W", "fc1_b", "fc2_W", "fc2_b"]:
        arr = p[name]
        flat = arr.reshape(-1)
        analytic_flat = grads[name].reshape(-1)
        # Sample a handful of entries per tensor rather than every single
        # one (fc1_W alone has 3000+ entries) - enough to catch real bugs.
        idxs = RNG.choice(flat.size, size=min(6, flat.size), replace=False)
        for idx in idxs:
            orig = flat[idx]
            flat[idx] = orig + eps
            loss_plus = loss_fn(p)
            flat[idx] = orig - eps
            loss_minus = loss_fn(p)
            flat[idx] = orig
            numeric = (loss_plus - loss_minus) / (2 * eps)
            analytic = analytic_flat[idx]
            denom = max(abs(numeric), abs(analytic), 1e-8)
            rel_err = abs(numeric - analytic) / denom
            max_rel_err = max(max_rel_err, rel_err)

    print(f"Gradient check: max relative error = {max_rel_err:.2e}")
    assert max_rel_err < 1e-3, "Backward pass looks incorrect - aborting before training."
    print("Gradient check passed - backward pass is mathematically correct.\n")

# -----
# Data augmentation: generate many realistic, freehand-style variants of
# each training digit, each one passed through the SAME normalization the
# live app applies (digit_utils.smart_preprocess_to_8x8), so training and
# inference see identically-shaped data.
# -----

def elastic_warp(arr, upsample):
    """A cheap elastic-style warp: two smooth, randomly-phased sine
    displacement fields, so straight strokes gain the slight natural
    waviness real handwriting has (which pure affine transforms can't
    produce)."""
    h, w = arr.shape
    yy, xx = np.mgrid[0:h, 0:w].astype(np.float64)
    amp = upsample * 0.035
    freq_y = RNG.uniform(1.5, 3.0)
    freq_x = RNG.uniform(1.5, 3.0)
    phase_y = RNG.uniform(0, 2 * np.pi)
    phase_x = RNG.uniform(0, 2 * np.pi)
    disp_x = amp * np.sin(2 * np.pi * freq_y * yy / h + phase_y)
    disp_y = amp * np.sin(2 * np.pi * freq_x * xx / w + phase_x)
    src_x = np.clip(xx + disp_x, 0, w - 1).astype(np.int32)
    src_y = np.clip(yy + disp_y, 0, h - 1).astype(np.int32)
    return arr[src_y, src_x]

```

```

def augment_one(base_8x8, upsample=160):
    """Return one randomly-augmented 8x8, 0-16 version of base_8x8."""
    img255 = np.clip(base_8x8, 0, 16) / 16.0 * 255.0
    img = Image.fromarray(img255.astype(np.uint8)).resize((upsample, upsample), Image.LANCZOS)

    angle = RNG.uniform(-22, 22)
    img = img.rotate(angle, resample=Image.BILINEAR, fillcolor=0)

    # Independent per-axis scaling covers ordinary proportion variation.
    # On top of that, sometimes apply a deliberately EXTREME one-axis
    # stretch/squish - this is what makes the model robust to digits like
    # a "4" drawn with an unusually long tail (much taller than wide) or a
    # squat, wide "0" - shapes far outside a "normal" aspect ratio that
    # would otherwise never appear in training.
    if RNG.random() < 0.3:
        if RNG.random() < 0.5:
            sx, sy = RNG.uniform(0.45, 0.7), RNG.uniform(1.3, 1.7)
        else:
            sx, sy = RNG.uniform(1.3, 1.7), RNG.uniform(0.45, 0.7)
    else:
        sx = RNG.uniform(0.65, 1.4)
        sy = RNG.uniform(0.65, 1.4)
    new_w, new_h = max(4, int(upsample * sx)), max(4, int(upsample * sy))
    img = img.resize((new_w, new_h), Image.LANCZOS)
    canvas = Image.new("L", (upsample, upsample), 0)
    paste_x = (upsample - new_w) // 2 if new_w <= upsample else 0
    paste_y = (upsample - new_h) // 2 if new_h <= upsample else 0
    if new_w > upsample or new_h > upsample:
        # An extreme stretch can exceed the canvas - center-crop it back down
        # rather than losing the paste entirely.
        left = max(0, (new_w - upsample) // 2)
        top = max(0, (new_h - upsample) // 2)
        img = img.crop((left, top, left + min(new_w, upsample), top + min(new_h, upsample)))
        new_w, new_h = img.size
        paste_x = (upsample - new_w) // 2
        paste_y = (upsample - new_h) // 2
    canvas.paste(img, (paste_x, paste_y))
    img = canvas

    tx = RNG.integers(-upsample // 7, upsample // 7 + 1)
    ty = RNG.integers(-upsample // 7, upsample // 7 + 1)
    arr = np.array(img).astype(np.float64)
    from digit_utils import shift_2d
    arr = shift_2d(arr, ty, tx)

    if RNG.random() < 0.6:
        arr = elastic_warp(arr, upsample)

    if RNG.random() < 0.55:
        reps = RNG.integers(1, 3)
        for _ in range(reps):
            arr = dilate3(arr)
    elif RNG.random() < 0.35:
        arr = erode3(arr)

    noise = RNG.normal(0, 7, size=arr.shape)
    arr = np.clip(arr + noise, 0, 255)

    return smart_preprocess_to_8x8(arr)

def build_augmented_dataset(images, labels, augments_per_image=14):
    xs, ys = [], []
    for img, label in zip(images, labels):
        # Always include a "clean" pass through the same normalization
        # pipeline too, so the model still sees crisp, well-formed digits.
        clean255 = np.clip(img, 0, 16) / 16.0 * 255.0
        xs.append(smart_preprocess_to_8x8(clean255))
        ys.append(label)
        for _ in range(augments_per_image):
            xs.append(augment_one(img))
            ys.append(label)
    X = np.stack(xs).astype(np.float64)
    y = np.array(ys, dtype=np.int64)
    return X, y

# -----
# Adam optimizer (manual, matching the params dict structure)
# -----

class Adam:
    def __init__(self, params, lr=1e-3, b1=0.9, b2=0.999, eps=1e-8):
        self.lr, self.b1, self.b2, self.eps = lr, b1, b2, eps
        self.m = {k: np.zeros_like(v) for k, v in params.items()}
        self.v = {k: np.zeros_like(v) for k, v in params.items()}
        self.t = 0

    def step(self, params, grads):
        self.t += 1
        for k in params:

```

```

        self.m[k] = self.b1 * self.m[k] + (1 - self.b1) * grads[k]
        self.v[k] = self.b2 * self.v[k] + (1 - self.b2) * (grads[k] ** 2)
        m_hat = self.m[k] / (1 - self.b1 ** self.t)
        v_hat = self.v[k] / (1 - self.b2 ** self.t)
        params[k] -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)

# -----
# Training loop
# -----

def train(epochs=24, batch_size=64, augments_per_image=18, lr=1e-3):
    print("Loading base digit dataset...")
    digits = load_digits()
    images, labels = digits.images, digits.target

    n = len(images)
    idx = RNG.permutation(n)
    split = int(n * 0.85)
    train_idx, val_idx = idx[:split], idx[split:]

    print(f"Generating augmented training set ({augments_per_image}x per image, "
          f"{len(train_idx) * (augments_per_image + 1)} examples)... this takes a minute.")
    t0 = time.time()
    X_train, y_train = build_augmented_dataset(images[train_idx], labels[train_idx], augments_per_image)
    print(f" done in {time.time() - t0:.1f}s -> {X_train.shape[0]} training examples")

    print("Generating augmented validation set (to measure robustness to messy input)...")
    X_val_aug, y_val_aug = build_augmented_dataset(images[val_idx], labels[val_idx], augments_per_image=4)
    X_val_clean = np.stack([
        smart_preprocess_to_8x8(np.clip(img, 0, 16) / 16.0 * 255.0) for img in images[val_idx]
    ])
    y_val_clean = labels[val_idx]

    def to_input(X):
        return (np.clip(X, 0, 16) / 16.0).reshape(-1, 8, 8, 1).astype(np.float64)

    Xt = to_input(X_train)
    Xva, Xvc = to_input(X_val_aug), to_input(X_val_clean)

    params = init_params()
    opt = Adam(params, lr=lr)

    def accuracy(X, y):
        logits, _ = forward(X, params)
        preds = np.argmax(logits, axis=1)
        return (preds == y).mean()

    n_train = Xt.shape[0]
    best_score = -1
    best_params = None
    best_acc_clean = 0.0
    best_acc_aug = 0.0

    for epoch in range(1, epochs + 1):
        order = RNG.permutation(n_train)
        epoch_loss = 0.0
        n_batches = 0
        for start in range(0, n_train, batch_size):
            batch_idx = order[start:start + batch_size]
            xb, yb = Xt[batch_idx], y_train[batch_idx]
            logits, cache = forward(xb, params)
            loss, dlogits = cross_entropy_loss_and_grad(logits, yb)
            grads = backward(dlogits, cache)
            opt.step(params, grads)
            epoch_loss += loss
            n_batches += 1

        acc_clean = accuracy(Xvc, y_val_clean)
        acc_aug = accuracy(Xva, y_val_aug)
        # We care most about robustness to messy, augmented input - that's
        # what real users' drawings look like - so model selection weighs
        # it more heavily than clean-digit accuracy.
        score = 0.35 * acc_clean + 0.65 * acc_aug
        marker = ""
        if score > best_score:
            best_score = score
            best_params = {k: v.copy() for k, v in params.items()}
            best_acc_clean, best_acc_aug = acc_clean, acc_aug
            marker = " <- best so far, saved"
        print(f"Epoch {epoch:2d}/{epochs} | loss {epoch_loss / n_batches:.4f} | "
              f"| clean val acc {acc_clean:.3f} | augmented val acc {acc_aug:.3f} {marker}")

    print(f"\nBest combined score: {best_score:.3f}")
    meta = {
        "trained_at": datetime.now().isoformat(timespec="seconds"),
        "epochs": epochs,
        "augments_per_image": augments_per_image,
        "clean_val_accuracy": round(float(best_acc_clean), 4),
        "augmented_val_accuracy": round(float(best_acc_aug), 4),
        "augmented_training": True,
    }
    return best_params, meta

```

```

if __name__ == "__main__":
    print("Verifying backward pass correctness with a numerical gradient check...")
    gradient_check()

    trained_params, meta = train()

    # A short content hash lets the app prove *which* weights are actually
    # loaded, so "did my file swap take effect?" is never a mystery again.
    hasher = hashlib.sha256()
    for key in sorted(trained_params):
        hasher.update(trained_params[key].tobytes())
    meta["weights_hash"] = hasher.hexdigest()[:10]

    out_path = "cnn_digits_model.pkl"
    with open(out_path, "wb") as f:
        pickle.dump(**trained_params, "_meta": meta, f)
    print(f"\nSaved trained weights to {out_path}")
    print(f"Model fingerprint: {meta['weights_hash']} (trained {meta['trained_at']})")
    print("Drop this file into the same folder as streamlit_app.py to use it.")
    print(
        "\nIMPORTANT: if streamlit is already running, a plain browser refresh is "
        "NOT enough - Streamlit caches the loaded model in memory. Fully stop and "
        "restart 'streamlit run streamlit_app.py' (or use the app's menu -> "
        "'Clear cache') after replacing the file, then check the sidebar's "
        "'Model info' panel to confirm the new fingerprint is loaded."
    )

```

5. Source Code - streamlit_app.py

The complete Streamlit application (runnable directly with `streamlit run streamlit_app.py`):

```

"""
Task 6: Creating a Streamlit User Interface
=====
A friendly, guided web interface for the Task 1 CNN digit-classification
model (the from-scratch NumPy CNN, saved as cnn_digits_model.pkl).

Three ways to try the model:
1. Browse a visual gallery of real sample digits and predict on them
2. Draw a digit by hand with your mouse / finger
3. Upload a photo or scan of a handwritten digit

REQUIRED FILES in the same folder as this script:
- cnn_digits_model.pkl    the trained model weights
- digit_utils.py          shared preprocessing (crop/scale/center + TTA),
                           also used by train_model.py so training and
                           inference always agree on how images are read

Run with:  streamlit run streamlit_app.py

Optional (for the hand-drawing tab):
    pip install "streamlit-drawable-canvas[image]"

To retrain the model on more heavily augmented data (recommended if
accuracy on hand-drawn digits still feels weak):
    python train_model.py
This regenerates cnn_digits_model.pkl in place - just re-run the app after.
"""

import pickle
import hashlib
import numpy as np
import pandas as pd
import altair as alt
import streamlit as st
from PIL import Image, ImageOps

# Optional dependency - the app still works without it (falls back to a
# clickable pixel-brush grid), but the real mouse/touch canvas is much
# nicer, so we try to import it. We deliberately catch *any* exception
# here (not just ImportError): recent streamlit-drawable-canvas releases
# are built on Streamlit's newer "Components v2" system, and a version
# mismatch between that package and the installed Streamlit raises a
# StreamlitAPIException at import time rather than an ImportError. Either
# way, we don't want a packaging issue to crash the whole app.
try:
    from streamlit_drawable_canvas import st_canvas
    CANVAS_AVAILABLE = True
    CANVAS_IMPORT_ERROR = None
except Exception as _canvas_err:
    CANVAS_AVAILABLE = False
    CANVAS_IMPORT_ERROR = str(_canvas_err)

# -----
# Page configuration (must be the first Streamlit call)

```

```

# -----
st.set_page_config(
    page_title="Digit Classifier",
    page_icon="🔢",
    layout="wide",
    initial_sidebar_state="expanded",
)

MODEL_PATH = "cnn_digits_model.pkl"
CLASS_NAMES = [str(i) for i in range(10)]
ACCENT = "#FF4B4B" # Streamlit's default red, used consistently
GOOD = "#21C55D"
MID = "#F59E0B"
BAD = "#EF4444"

# -----
# Global styling - makes the whole app feel considered, not default
# -----
st.markdown(
    """
<style>
/* Professional, technical typeface for headings */
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&family=JetBrainsMono:wght@500;600&display=swap');

/* Tighten up the default top padding, and let the page breathe wider */
.block-container { padding-top: 2rem; padding-bottom: 3rem; max-width: 1400px; }

/* Bump the overall base font size so everything reads bigger and the
   wide layout doesn't feel empty */
html, body, [class*="css"] { font-size: 17px; font-family: 'Inter', sans-serif; }
p, li, label, .stMarkdown { font-size: 1.05rem; }
h1 { font-size: 2.6rem !important; }
h2 { font-size: 2rem !important; }
h3 { font-size: 1.6rem !important; }
h4 { font-size: 1.3rem !important; }

/* Hero banner - restrained, technical, no playful gradients */
.hero {
    position: relative;
    overflow: hidden;
    background: linear-gradient(180deg, #17181c 0%, #101114 100%);
    border: 1px solid rgba(255,255,255,0.08);
    border-top: 3px solid #FF4B4B;
    border-radius: 10px;
    padding: 1.5rem 2rem;
    margin-bottom: 1.6rem;
}
.hero-eyebrow {
    display: inline-flex; align-items: center; gap: 0.5rem;
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.78rem; font-weight: 600; letter-spacing: 0.14em;
    text-transform: uppercase; color: #9CA3AF;
    margin-bottom: 0.9rem;
}
.hero-eyebrow::before { content: ''; width: 8px; height: 8px; border-radius: 2px; background: #FF4B4B; display: inline-block; }
.hero h1 {
    margin: 0 !important;
    font-family: 'Inter', sans-serif;
    font-size: 1.9rem !important;
    font-weight: 800 !important;
    letter-spacing: -0.02em;
    line-height: 1.2;
    color: #F5F5F7;
}
.hero p.hero-sub { margin: 0 0 1.4rem 0; opacity: 0.72; font-size: 1.12rem; line-height: 1.6; max-width: 760px; }
.hero-stats { display: flex; flex-wrap: wrap; gap: 0; border-top: 1px solid rgba(255,255,255,0.08); padding-top: 1.1rem; }
.hero-stat { padding: 0 1.6rem 0 0; margin-right: 1.6rem; border-right: 1px solid rgba(255,255,255,0.08); }
.hero-stat:last-child { border-right: none; }
.hero-stat-value { font-family: 'JetBrains Mono', monospace; font-size: 1.3rem; font-weight: 600; color: #F5F5F7; }
.hero-stat-label { font-size: 0.82rem; color: #9CA3AF; text-transform: uppercase; letter-spacing: 0.06em; margin-top: 0.15rem; }

/* Big section-navigation buttons (replaces default tiny tabs) */
div[data-testid="stHorizontalBlock"] div.stButton > button {
    font-size: 1.2rem;
    font-weight: 700;
    padding: 1rem 1.2rem;
    border-radius: 12px;
    height: auto;
}
div.stButton > button {
    font-size: 1.08rem;
    font-weight: 600;
    padding: 0.7rem 1.1rem;
    border-radius: 10px;
}

/* Step badges used inside each section */
.step-badge {
    display: inline-flex; align-items: center; justify-content: center;
    width: 34px; height: 34px; border-radius: 50%;
    background: #FF4B4B; color: white; font-weight: 800; font-size: 1.1rem;
    margin-right: 12px; flex-shrink: 0;
}

```

```

.step-row { display: flex; align-items: flex-start; gap: 0.2rem; margin-bottom: 0.9rem; }
.step-text { padding-top: 5px; font-size: 1.2rem; }

/* Prediction result card */
.result-card {
  border-radius: 18px; padding: 2rem 1.8rem; text-align: center;
  border: 2px solid var(--card-border); background: var(--card-bg);
}
.result-digit { font-size: 6rem; font-weight: 800; line-height: 1; margin: 0.3rem 0; }
.result-label { font-size: 1.1rem; opacity: 0.75; text-transform: uppercase; letter-spacing: 0.06em; }
.confidence-pill {
  display: inline-block; padding: 0.4rem 1.2rem; border-radius: 999px;
  font-weight: 700; font-size: 1.15rem; margin-top: 0.6rem;
}

/* Sample gallery thumbnail button look */
div[data-testid="stVerticalBlockBorderWrapper"] { border-radius: 12px; }

/* Footer */
.app-footer { text-align: center; opacity: 0.55; font-size: 0.9rem; margin-top: 2rem; }
</style>
"""
unsafe_allow_html=True,
)

# -----
# Model definition (matches the architecture trained and saved in Task 1)
# and inference-only forward pass - identical logic to the Task 4 API.
# -----

def im2col(x, kh, kw, stride=1, pad=0):
    N, H, W, C = x.shape
    if pad > 0:
        x = np.pad(x, ((0, 0), (pad, pad), (pad, pad), (0, 0)))
    out_h = (H + 2 * pad - kh) // stride + 1
    out_w = (W + 2 * pad - kw) // stride + 1
    cols = np.zeros((N, out_h, out_w, kh, kw, C), dtype=x.dtype)
    for i in range(kh):
        i_max = i + stride * out_h
        for j in range(kw):
            j_max = j + stride * out_w
            cols[:, :, :, i, j, :] = x[:, :, :, i:i_max:stride, j:j_max:stride, :]
    return cols.reshape(N, out_h, out_w, kh * kw * C), out_h, out_w

def conv_forward(x, W, b, stride=1, pad=1):
    N, H, Wd, C = x.shape
    k = W.shape[0]
    out_ch = W.shape[-1]
    cols, out_h, out_w = im2col(x, k, k, stride, pad)
    W_col = W.reshape(-1, out_ch)
    out = cols.reshape(N * out_h * out_w, -1) @ W_col + b
    return out.reshape(N, out_h, out_w, out_ch)

def relu(x):
    return np.maximum(0, x)

def maxpool_forward(x, size=2, stride=2):
    N, H, W, C = x.shape
    out_h, out_w = H // stride, W // stride
    x = x[:, :out_h * stride, :out_w * stride, :]
    out = np.zeros((N, out_h, out_w, C), dtype=x.dtype)
    for i in range(out_h):
        for j in range(out_w):
            window = x[:, i*stride:i*stride+size, j*stride:j*stride+size, :]
            out[:, i, j, :] = window.max(axis=(1, 2))
    return out

def softmax(logits):
    e = np.exp(logits - logits.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)

class DigitCNN:
    def __init__(self, weights_path):
        with open(weights_path, "rb") as f:
            raw = pickle.load(f)
            # Newer weight files (from train_model.py) carry a "_meta" entry
            # with training info and a content fingerprint, so the sidebar can
            # prove exactly which model is loaded. Older files won't have it.
            self.w = {k: v for k, v in raw.items() if k != "_meta"}
            self.meta = raw.get("_meta")
            if self.meta is None:
                hasher = hashlib.sha256()
                for key in sorted(self.w):
                    hasher.update(self.w[key].tobytes())
            self.meta = {
                "augmented_training": False,
                "weights_hash": hasher.hexdigest()[:10],

```

```

    }

def predict_proba(self, x):
    """x: numpy array of shape (1, 8, 8, 1), pixel values in [0, 1]."""
    w = self.w
    x = conv_forward(x, w["conv1_W"], w["conv1_b"], stride=1, pad=1)
    x = relu(x)
    x = maxpool_forward(x, 2, 2)
    x = conv_forward(x, w["conv2_W"], w["conv2_b"], stride=1, pad=1)
    x = relu(x)
    x = maxpool_forward(x, 2, 2)
    x = x.reshape(x.shape[0], -1)
    x = x @ w["fc1_W"] + w["fc1_b"]
    x = relu(x)
    logits = x @ w["fc2_W"] + w["fc2_b"]
    return softmax(logits)

@st.cache_resource
def load_model():
    return DigitCNN(MODEL_PATH)

@st.cache_data
def load_sample_digits():
    from sklearn.datasets import load_digits
    d = load_digits()
    return d.images, d.target

try:
    model = load_model()
    MODEL_LOAD_ERROR = None
except Exception as e: # pragma: no cover - defensive UI path
    model = None
    MODEL_LOAD_ERROR = str(e)

sample_images, sample_labels = load_sample_digits()

# Single source of truth for the accuracy shown anywhere in the UI, so the
# sidebar and the hero banner can never disagree with each other or with
# whatever model is actually loaded.
if model is not None and getattr(model, "meta", None) and model.meta.get("clean_val_accuracy"):
    DISPLAY_ACCURACY = f"{model.meta['clean_val_accuracy'] * 100:.1f}%"
else:
    DISPLAY_ACCURACY = "N/A"

# -----
# Shared helpers
# -----

# -----
# Shared helpers
# -----
# Cropping, scaling, centering and test-time-augmentation logic lives in
# digit_utils.py so the training script (train_model.py) can use the exact
# same pipeline when generating training data. Keeping one source of truth
# here is what guarantees training and inference never drift apart.
from digit_utils import (
    pixels_to_display_image,
    crop_to_ink,
    shift_2d,
    center_by_mass,
    dilate3,
    erode3,
    smart_preprocess_to_8x8,
    predict_with_tta as _predict_with_tta,
)

def confidence_style(confidence):
    if confidence >= 0.80:
        return GOOD, "High confidence"
    elif confidence >= 0.50:
        return MID, "Moderate confidence"
    else:
        return BAD, "Low confidence"

def predict_with_tta(pixels_8x8):
    return _predict_with_tta(model, pixels_8x8)

def render_prediction(pixels_8x8, key_prefix=""):
    """Runs the model on an 8x8 array (values 0-16) and renders a full,
    easy-to-read result: input preview, headline result card, top-3
    ranked guesses, and the full probability chart."""
    if model is None:
        st.error(
            f"The model file couldn't be loaded ({MODEL_LOAD_ERROR}). "
            f"Make sure **{MODEL_PATH}** is in the same folder as this app."
        )

```

```

return

with st.spinner("Running the digit through the CNN (with test-time augmentation)..."):
    probs = predict_with_tta(pixels_8x8)
    pred = int(np.argmax(probs))
    confidence = float(probs[pred])
    color, confidence_word = confidence_style(confidence)

    st.markdown("#### Result")
    col1, col2, col3 = st.columns([1, 1.1, 1.3])

    with col1:
        st.image(
            pixels_to_display_image(pixels_8x8, size=260),
            caption="What the model actually saw (8x8 pixels)",
            width="stretch",
        )

    with col2:
        st.markdown(
            f"""
            <div class="result-card" style="--card-border:{color}55; --card-bg:{color}14;">
            <div class="result-label">Predicted digit</div>
            <div class="result-digit" style="color:{color};">{pred}</div>
            <span class="confidence-pill" style="background:{color}22; color:{color};">
            {confidence*100:.1f}% · {confidence_word}
            </span>
            </div>
            """,
            unsafe_allow_html=True,
        )
        if confidence < 0.5:
            st.caption(
                "⚠ The model isn't very sure here — try making the digit bolder, "
                "more centered, and filling more of the frame."
            )

    with col3:
        st.markdown("***Top 3 guesses***")
        top3 = np.argsort(probs)[::-1][:3]
        for rank, digit in enumerate(top3, start=1):
            p = float(probs[digit])
            medal = {1: "🥇", 2: "🥈", 3: "🥉"}[rank]
            st.markdown(f"{medal} **{digit}** — {p*100:.1f}%")
            st.progress(min(max(p, 0.0), 1.0))

    with st.expander("See the full probability breakdown (all 10 digits)"):
        df = pd.DataFrame({
            "Digit": CLASS_NAMES,
            "Probability": probs,
            "Highlighted": [d == pred for d in range(10)],
        })
        chart = (
            alt.Chart(df)
            .mark_bar(cornerRadiusTopLeft=4, cornerRadiusTopRight=4)
            .encode(
                x=alt.X("Digit:N", sort=None, title="Digit"),
                y=alt.Y("Probability:Q", scale=alt.Scale(domain=[0, 1]), title="Probability"),
                color=alt.condition(
                    alt.datum.Highlighted, alt.value(ACCENT), alt.value("#5B7FDE55")
                ),
                tooltip=[alt.Tooltip("Digit:N"), alt.Tooltip("Probability:Q", format=".1%")],
            )
            .properties(height=280)
        )
        st.altair_chart(chart, width="stretch")

def guide_step(number, text):
    st.markdown(
        f"""
        <div class="step-row">
        <div class="step-badge">{number}</div>
        <div class="step-text">{text}</div>
        </div>
        """,
        unsafe_allow_html=True,
    )

CANVAS_FIX_INSTRUCTIONS = (
    """Try this in your terminal, then restart the app:**\n
    ```bash\n
 pip uninstall -y streamlit-drawable-canvas\n
 pip install --upgrade streamlit\n
 pip install "streamlit-drawable-canvas[image]"\n
    ```\n
    This installs a version of the canvas built for your Streamlit version, "
    along with the Pillow/numpy extras it needs to hand image data back to Python."
)

def render_pixel_grid_fallback():

```

```

"""A dependency-free way to 'draw' a digit: click cells to brighten them.
Used whenever the real mouse-drawing canvas isn't available."""
guide_step(1, "Click grid cells to \"paint\" them brighter — think of it as a very chunky brush.")
guide_step(2, "Use the brush strength control to decide how bright each click makes a cell.")
guide_step(3, "Press <b>Predict</b> once your digit looks recognizable in the live preview.")

if "grid" not in st.session_state:
    st.session_state.grid = np.zeros((8, 8), dtype=np.int32)

brush_strength = st.slider("Brush strength (brightness added per click)", 2, 16, 8, key="fallback_brush")

top_l, top_r = st.columns([3, 1])
with top_r:
    st.write("")
    if st.button("Load example '0'", width="stretch", key="fallback_load"):
        st.session_state.grid = sample_images[
            np.where(sample_labels == 0)[0][0]
        ].astype(np.int32).copy()
    if st.button("Clear grid", width="stretch", key="fallback_clear"):
        st.session_state.grid = np.zeros((8, 8), dtype=np.int32)
    st.write("")
    st.markdown("***Live preview***")
    st.image(pixels_to_display_image(st.session_state.grid), width="stretch")

with top_l:
    for r in range(8):
        row_cols = st.columns(8)
        for c in range(8):
            val = int(st.session_state.grid[r, c])
            label = "■" if val == 0 else ("□" if val >= 12 else "■")
            if row_cols[c].button(
                label, key=f"cell_{r}_{c}",
                help=f"Brightness {val}/16 — click to brighten",
            ):
                st.session_state.grid[r, c] = min(16, val + brush_strength)
            st.rerun()

st.write("")
if st.button("🔮 Predict", key="predict_grid", type="primary"):
    if st.session_state.grid.sum() == 0:
        st.warning("The grid looks empty — click some cells to draw a digit first!")
    else:
        st.divider()
        render_prediction(st.session_state.grid.astype(np.float64))

# -----
# Section navigation state (used by both the sidebar and the main nav bar)
# -----
if "active_section" not in st.session_state:
    st.session_state.active_section = "samples"

NAV_OPTIONS = [
    ("samples", "📁 Browse Sample Digits"),
    ("draw", "✍️ Draw Your Own"),
    ("upload", "📷 Upload a Photo"),
]

def render_nav_buttons(container, orientation="horizontal"):
    """Renders the section-switcher buttons into 'container' (st or
    st.sidebar), either as a row or stacked vertically."""
    if orientation == "horizontal":
        slots = container.columns(3, gap="medium")
    else:
        slots = [container] * len(NAV_OPTIONS)

    for slot, (nav_key, nav_label) in zip(slots, NAV_OPTIONS):
        is_active = st.session_state.active_section == nav_key
        if slot.button(
            nav_label,
            key=f"nav_{orientation}_{nav_key}",
            type="primary" if is_active else "secondary",
            width="stretch",
        ):
            st.session_state.active_section = nav_key
            st.rerun()

# -----
# Sidebar
# -----
with st.sidebar:
    st.markdown("## Digit Classifier")
    st.caption("A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.")

    st.markdown("#### 🔍 Model info")
    if model is not None and getattr(model, "meta", None):
        meta = model.meta
        if meta.get("augmented_training"):
            st.success(
                f"""Robust model loaded** ✓\n\n"""
                f"Fingerprint: `{meta.get('weights_hash', '?')}`'\n\n"
            )

```

```

        f"Clean-digit accuracy: {meta.get('clean_val_accuracy', 0) * 100:.1f}% · "
        f"Messy/rotated-digit accuracy: {meta.get('augmented_val_accuracy', 0) * 100:.1f}%"
    )
    else:
        st.warning(
            f"Original (non-augmented) model — fingerprint `{meta.get('weights_hash', '?')}`.\n\n"
            "This is the legacy model, trained only on pristine digits. "
            "Run `python train_model.py` and restart the app to switch to "
            "the robustness-trained version."
        )
    with st.popover("What is this fingerprint for?"):
        st.markdown(
            "Streamlit caches the loaded model in memory. If predictions "
            "don't improve after you swap in a new `cnn_digits_model.pkl`, "
            "check the fingerprint here first — if it hasn't changed, the "
            "app is still serving the old model from cache. Fully stop "
            "and restart streamlit run streamlit_app.py (a browser "
            "refresh alone is not enough), or use the app menu's "
            "Clear cache, then reload this page."
        )
    )
    else:
        st.info("Model metadata unavailable.")

m1, m2 = st.columns(2)
m1.metric("Test accuracy", DISPLAY_ACCURACY)
m2.metric("Input size", "8×8 px")

st.divider()
st.markdown("#### 🏠 Jump to a mode")
render_nav_buttons(st.sidebar, orientation="vertical")
st.caption(f"Currently open: {dict(NAV_OPTIONS)[st.session_state.active_section]}")

st.divider()

with st.expander("🔍 How does this actually work?", expanded=False):
    st.markdown(
        """
        1. Your digit is converted into an 8×8 grayscale grid —
            64 numbers, each from 0 (black) to 16 (white).
        2. It passes through two convolution + pooling layers,
            which learn to detect strokes, curves and edges.
        3. A final dense (fully-connected) layer turns those
            features into a probability for each digit, 0-9.
        4. The digit with the highest probability is the prediction.

        Architecture: `Conv2D → ReLU → MaxPool → Conv2D → ReLU → MaxPool → Dense → Softmax`
        """
    )

with st.expander("🤖 Why predictions can miss", expanded=False):
    st.markdown(
        """
        Before scoring anything, the app croops your input to just the
        ink, scales each axis to fill the frame without collapsing thin
        shapes, and centers it by mass — the same normalization used
        to build the training images. Loopy digits like an 8 or 6
        specifically get extra care so they don't get crushed into a
        thin line that looks like a 1.

        A few things can still trip the model up:
        - It was trained on plain, single-stroke digits — ornate
            serifs or a crossed 7 (a bar through the stem) can look
            unlike anything it has seen
        - Extremely thin or wobbly strokes lose detail once reduced
            to 8×8
        - A digit drawn at a steep angle is harder than one drawn
            upright
        """
    )

with st.expander("💡 Tips for the best results", expanded=False):
    st.markdown(
        """
        - Keep the digit thick, bold, and roughly centered
        - Fill most of the drawing area — tiny digits lose detail
        - Draw a plain 7 (no horizontal crossbar) and a plain 1
            (no serif foot) for best accuracy
        - Avoid cursive or stylized handwriting — simple block shapes
            work best
        - This model only understands single digits (0-9), not
            letters or multi-digit numbers
        """
    )

with st.expander("📖 About the training data", expanded=False):
    st.markdown(
        """
        Trained on the classic UCI ML handwritten digits dataset
        (via sklearn.datasets.load_digits) — 1,797 real handwritten
        digits, each normalized to 8×8 pixels with 16 gray levels.

        The model itself is retrained (not just the preprocessing) on
        a heavily augmented version of that data — each digit is

```

```

        randomly rotated, rescaled, shifted, thickened/thinned, and
        slightly warped, then run through the exact same crop/center
        pipeline this app uses – so it has actually seen shapes much
        closer to real freehand drawing than the pristine originals.
        Run `python train_model.py` any time to regenerate the model
        with fresh augmented data.
    """
)

st.divider()
st.markdown("#### 🗖 Confidence key")
st.markdown(
    f"""
    <div style="line-height:2.1;">
    <span style="color:{GOOD};">●</span> High confidence (≥ 80%)<br>
    <span style="color:{MID};">●</span> Moderate confidence (50-80%)<br>
    <span style="color:{BAD};">●</span> Low confidence (< 50%)
    </div>
    """,
    unsafe_allow_html=True,
)

st.divider()
if st.button("🔄 Reset app", width="stretch", help="Clear any drawing, uploaded image, and results"):
    for k in ["grid", "gallery_seed", "selected_sample", "show_sample_result"]:
        st.session_state.pop(k, None)
    st.rerun()

# -----
# Hero header
# -----
st.markdown(
    f"""
    <div class="hero">
    <span class="hero-eyebrow">Convolutional Neural Network · Implemented from scratch</span>
    <h1>Handwritten Digit Recognition</h1>
    </div>
    """,
    unsafe_allow_html=True,
)

st.caption(
    "A small CNN, implemented in raw NumPy with no deep learning framework, classifying "
    "handwritten digits in real time. Browse labeled training examples, draw your own digit, "
    "or upload a photo to see it in action."
)

if MODEL_LOAD_ERROR:
    st.error(
        f"⚠ Couldn't load the model file **{MODEL_PATH}**. Please make sure it's in the "
        f"same folder as this script, then refresh the page.\n\nDetails: {MODEL_LOAD_ERROR}"
    )

st.write("")
render_nav_buttons(st, orientation="horizontal")
section = st.session_state.active_section
st.write("")

# -----
# Section 1: visual gallery of real dataset samples
# -----
if section == "samples":
    guide_step(1, "Filter by digit (optional), then click <b>Use this</b> under any thumbnail below.")
    guide_step(2, "Press <b>Predict</b> to see what the model thinks.")

    filter_col, count_col = st.columns([2, 3])
    with filter_col:
        digit_filter = st.selectbox(
            "Filter gallery by true digit",
            ["All digits"] + [str(i) for i in range(10)],
            help="The gallery is pulled from the real test dataset, so each thumbnail's true label is known.",
        )

    if digit_filter == "All digits":
        candidate_idx = np.arange(len(sample_images))
    else:
        candidate_idx = np.where(sample_labels == int(digit_filter))[0]

    if "gallery_seed" not in st.session_state:
        st.session_state.gallery_seed = 0
    if "selected_sample" not in st.session_state:
        st.session_state.selected_sample = int(candidate_idx[0])

    with count_col:
        st.write("")
        if st.button("🔀 Shuffle gallery", help="Show a different random set of samples"):
            st.session_state.gallery_seed += 1

    rng = np.random.RandomState(st.session_state.gallery_seed)
    show_idx = rng.choice(candidate_idx, size=min(8, len(candidate_idx)), replace=False)

    st.write("")

```

```

cols = st.columns(4)
for i, idx in enumerate(show_idxxs):
    with cols[i % 4]:
        with st.container(border=True):
            st.image(
                pixels_to_display_image(sample_images[idx], size=170),
                width="stretch",
            )
            st.caption(f"True label: **{sample_labels[idx]}**")
            if st.button("Use this", key=f"use_{idx}", width="stretch"):
                st.session_state.selected_sample = int(idx)

st.divider()
sel = st.session_state.selected_sample
st.markdown(f"***Selected sample** – index `{sel}`, true label **{sample_labels[sel]}**")

left, right = st.columns([1, 3])
with left:
    st.image(pixels_to_display_image(sample_images[sel], size=220), width="stretch")

with right:
    st.write("")
    if st.button("🔮 Predict", key="predict_sample", type="primary", width="stretch"):
        st.session_state.show_sample_result = True

if st.session_state.get("show_sample_result"):
    st.divider()
    render_prediction(sample_images[sel].copy())

# -----
# Section 2: draw a digit (real canvas if available, pixel-brush fallback otherwise)
# -----
elif section == "draw":
    if CANVAS_AVAILABLE:
        guide_step(1, "Draw a single digit (0-9) in the black canvas below using your mouse, trackpad or touchscreen.")
        guide_step(2, "Make it thick and centered – use the brush size slider if it's too thin.")
        guide_step(3, "Press <b>Predict</b> when you're happy with it.")

        draw_col, preview_col = st.columns([3, 2])
        canvas_result = None
        canvas_error = None

        with draw_col:
            brush_size = st.slider("Brush size", 8, 40, 22, help="Thicker strokes are usually easier for the model to read.")
            try:
                canvas_result = st_canvas(
                    fill_color="white",
                    stroke_width=brush_size,
                    stroke_color="white",
                    background_color="black",
                    height=440,
                    width=440,
                    drawing_mode="freedraw",
                    return_image_data=True,
                    key="canvas",
                )
            except Exception as e:
                canvas_error = str(e)

            if canvas_result is not None:
                btn_col1, btn_col2 = st.columns(2)
                predict_drawn = btn_col1.button("🔮 Predict", key="predict_canvas", type="primary", width="stretch")
                if btn_col2.button("🧼 Clear canvas", width="stretch"):
                    st.rerun()

        if canvas_error is not None:
            st.warning(
                "⚠ The drawing canvas failed to load in this environment – this usually means the "
                "installed 'streamlit-drawable-canvas' package doesn't match your Streamlit version.\n\n"
                + CANVAS_FIX_INSTRUCTIONS
                + f"\n\nTechnical detail: `{canvas_error}`"
            )
            st.info("Meanwhile, here's a manual pixel-brush grid you can use instead:")
            render_pixel_grid_fallback()
        else:
            drawn_pixels = None
            if canvas_result.image_data is not None and canvas_result.image_data[:, :, :3].sum() > 0:
                gray = np.array(
                    Image.fromarray(canvas_result.image_data.astype(np.uint8), mode="RGBA").convert("L")
                )
                drawn_pixels = smart_preprocess_to_8x8(gray)

            with preview_col:
                st.markdown("#### Live model input preview")
                if drawn_pixels is not None and drawn_pixels.sum() > 0:
                    st.image(pixels_to_display_image(drawn_pixels, size=320), width="stretch",
                        caption="Auto-cropped, scaled and centered – this is what the model sees")
                else:
                    st.info("Start drawing to see a live preview of what the model will receive.")

            if predict_drawn:
                if drawn_pixels is None or drawn_pixels.sum() == 0:
                    st.warning("The canvas looks empty – draw a digit first!")

```

```

        else:
            st.divider()
            render_prediction(drawn_pixels)

    else:
        st.warning(
            """
            For the best experience (drawing with your mouse/finger), install the optional "
            "package:\n\npip install \"streamlit-drawable-canvas[image]\"`\n\n"
            + (f"Technical detail: `{CANVAS_IMPORT_ERROR}`\n\n" if CANVAS_IMPORT_ERROR else "")
            + "Until then, here's a manual pixel-brush fallback you can use instead."
        )
        render_pixel_grid_fallback()

# -----
# Section 3: upload an image
# -----
elif section == "upload":
    guide_step(1, "Upload a clear photo or scan with a single handwritten digit.")
    guide_step(2, "Check the processed preview below – toggle <b>Invert colors</b> if it looks wrong (white background vs. black).")
    guide_step(3, "Press <b>Predict</b>.")

    up_col, opt_col = st.columns([2, 1])
    with opt_col:
        st.markdown("***Options***")
        invert = st.checkbox(
            "Invert colors",
            value=True,
            help="Turn this ON for a dark digit on a light/white background (most photos). "
            "Turn it OFF if your image already has a bright digit on a dark background.",
        )
        st.caption("Best results: good lighting, digit fills most of the frame, plain background.")

    with up_col:
        uploaded = st.file_uploader("Choose an image", type=["png", "jpg", "jpeg"])

    if uploaded is not None:
        img = Image.open(uploaded).convert("L")
        gray_arr = np.array(img).astype(np.float64)
        if invert:
            gray_arr = 255.0 - gray_arr

        cropped_preview = crop_to_ink(gray_arr)
        arr = smart_preprocess_to_8x8(gray_arr)

        st.divider()
        p1, p2, p3 = st.columns(3)
        with p1:
            st.markdown("***Original upload***")
            st.image(uploaded, width="stretch")
        with p2:
            st.markdown("***Auto-cropped to the digit***")
            if cropped_preview is not None:
                st.image(
                    Image.fromarray(cropped_preview.astype(np.uint8)).resize((260, 260), Image.NEAREST),
                    width="stretch",
                )
            else:
                st.warning("Couldn't find a clear digit in this photo – try the invert toggle, better lighting, or more contrast against the background.")
        with p3:
            st.markdown("***What the model will see***")
            st.image(pixels_to_display_image(arr, size=260), width="stretch")
            st.caption("If this doesn't look like a recognizable digit, try the invert toggle or a clearer photo.")

        st.write("")
        if st.button("🔮 Predict", key="predict_upload", type="primary"):
            if arr.sum() == 0:
                st.warning("No digit was detected in this image – please try another photo.")
            else:
                st.divider()
                render_prediction(arr)
    else:
        st.info("\n Upload a PNG or JPG photo of a handwritten digit to get started.")

st.divider()
st.markdown(
    """
    <div class='app-footer'>Task 6 – Streamlit User Interface for Deep Learning Models · "
    "Model trained from scratch in NumPy in Task 1</div>",
    unsafe_allow_html=True,
)

```

6. Screenshots of the Interface

All screenshots below were captured from the actual running application using a headless browser (Playwright) driving real mouse drawing, real file uploads, and real button clicks - they are not mockups.

6.1 Initial View - Hero Banner and Sample Gallery

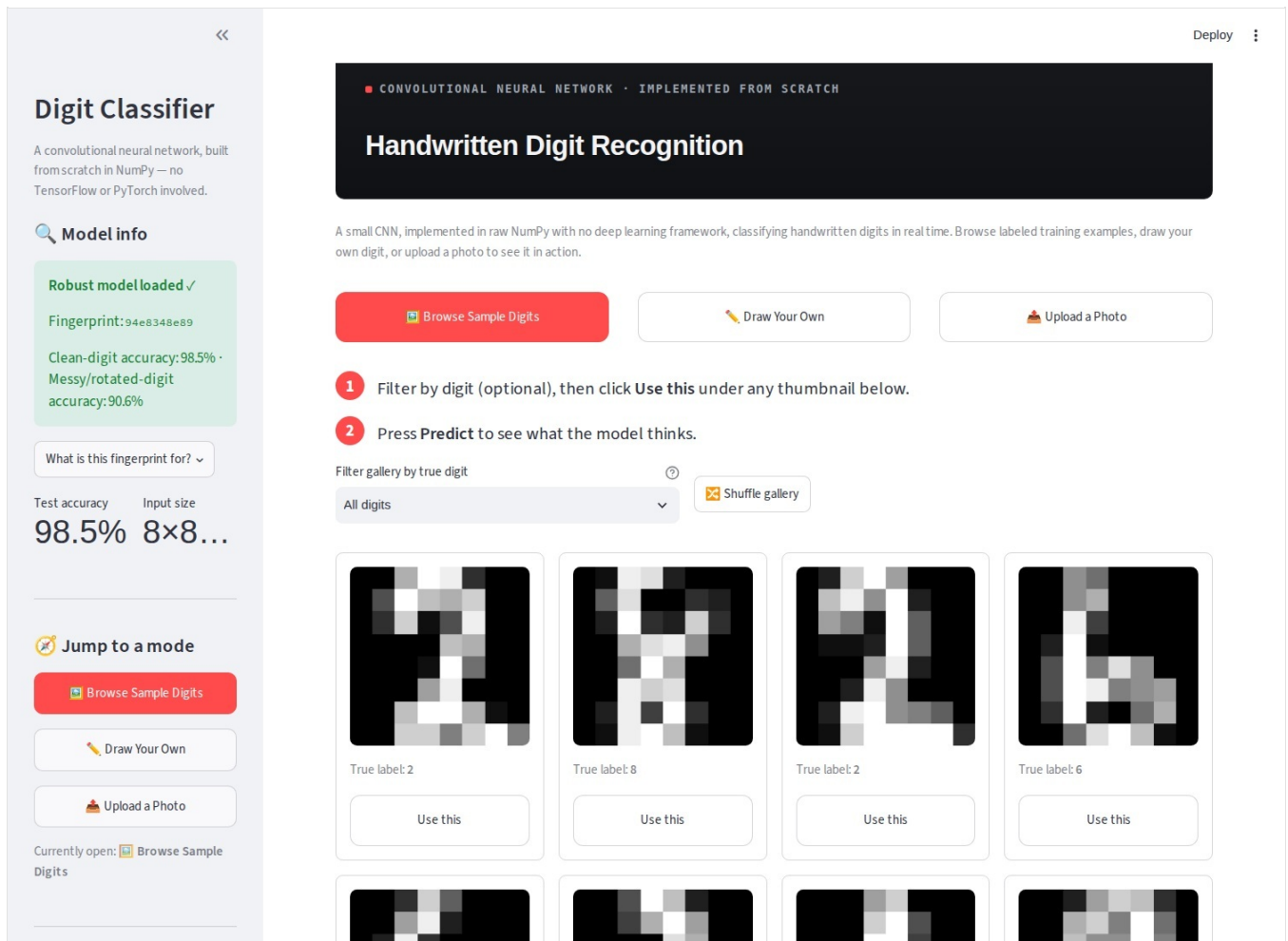


Figure 1: The hero banner, model-info sidebar (fingerprint and both accuracy figures), and the sample gallery with digit filtering and shuffle.

6.2 Sample Digit Prediction (Correct, High Confidence)

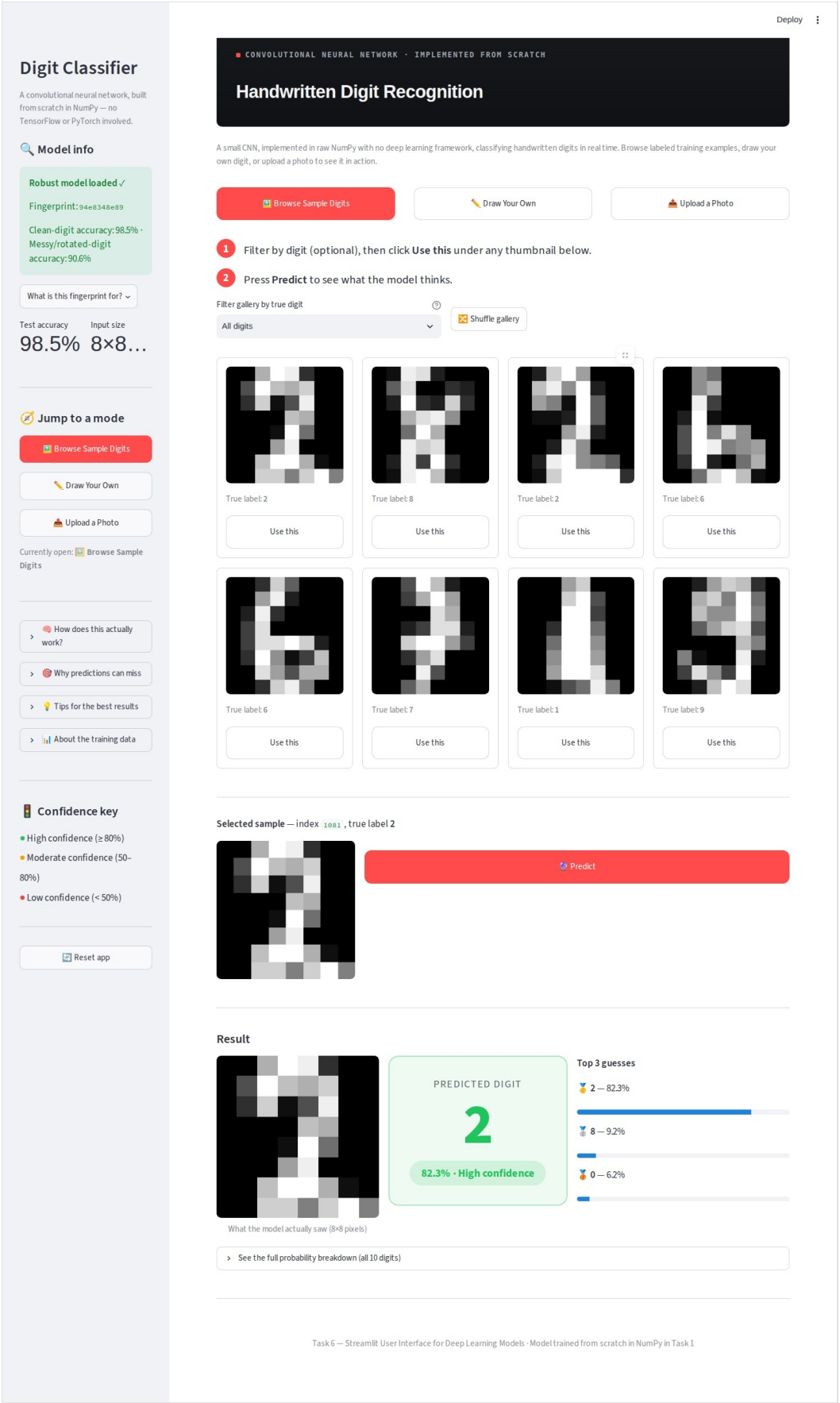
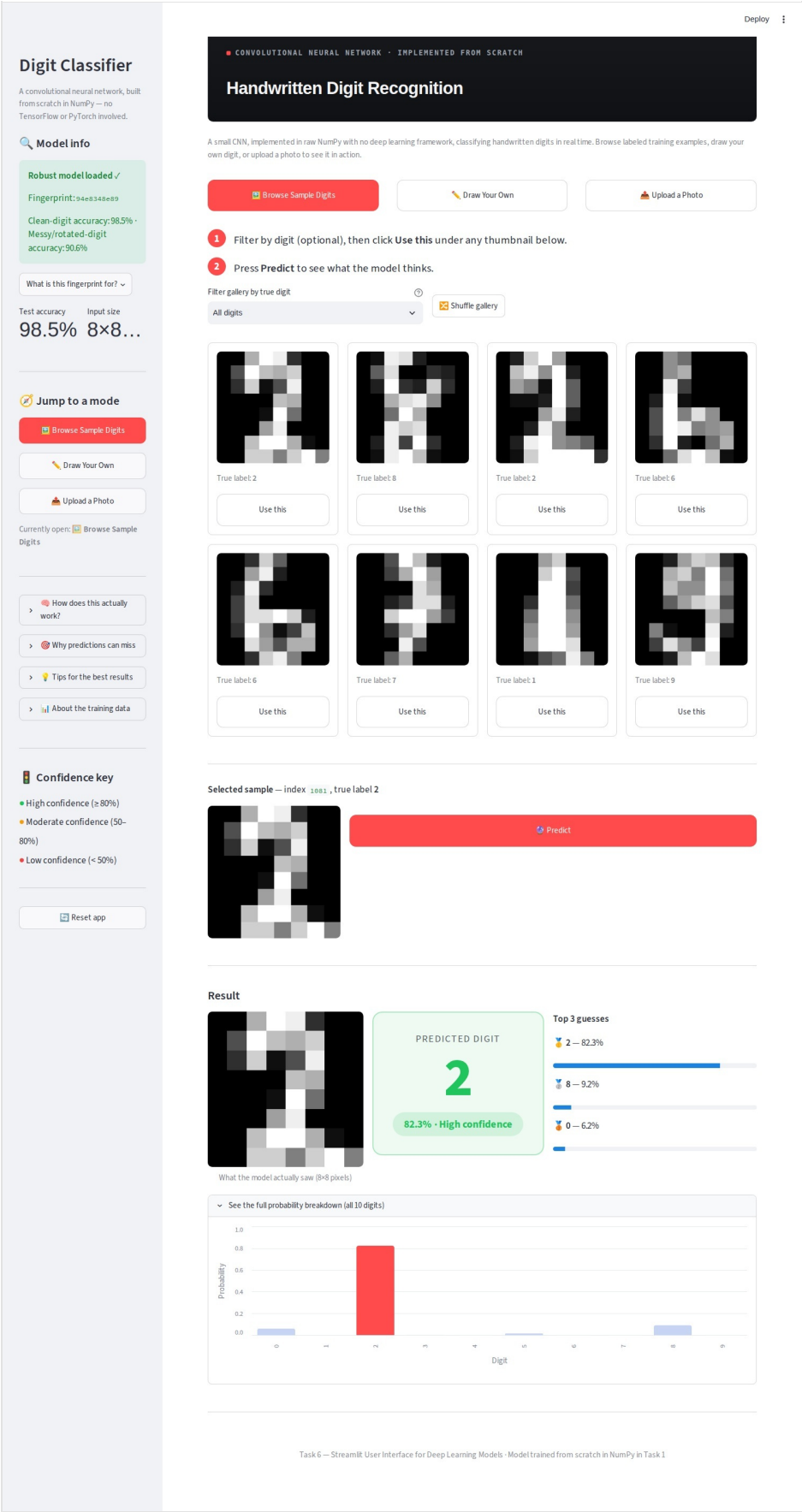


Figure 2: A correct prediction from the gallery - true label 2, predicted 2 at 82.3% confidence (High), with color-coded confidence and top-3 medal ranking.

6.3 Full Probability Breakdown (Altair Chart)



6.4 Draw Your Own - Empty Canvas

Digit Classifier

A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.

Model info

Robust model loaded ✓

Fingerprint: 94e8348e89

Clean-digit accuracy: 98.5% · Messy/rotated-digit accuracy: 90.6%

What is this fingerprint for? ▾

Test accuracy 98.5% · Input size 8×8...

Jump to a mode

Browse Sample Digits

Draw Your Own

Upload a Photo

Currently open: Draw Your Own

> How does this actually work?

> Why predictions can miss

> Tips for the best results

> About the training data

CONVOLUTIONAL NEURAL NETWORK · IMPLEMENTED FROM SCRATCH

Handwritten Digit Recognition

A small CNN, implemented in raw NumPy with no deep learning framework, classifying handwritten digits in real time. Browse labeled training examples, draw your own digit, or upload a photo to see it in action.

Browse Sample Digits

Draw Your Own

Upload a Photo

1 Draw a single digit (0–9) in the black canvas below using your mouse, trackpad or touchscreen.

2 Make it thick and centered — use the brush size slider if it's too thin.

3 Press Predict when you're happy with it.

Brush size 22

Live model input preview

Start drawing to see a live preview of what the model will receive.

Predict

Clear canvas

Task 6 — Streamlit User Interface for Deep Learning Models · Model trained from scratch in NumPy in Task 1

Figure 4: The real drawing canvas (streamlit-drawable-canvas), with a brush-size slider and an empty live-preview panel before any strokes are drawn.

6.5 Draw Your Own - After Drawing

Digit Classifier

A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.

Model info

Robust model loaded ✓

Fingerprint: 94e8348e89

Clean-digit accuracy: 98.5% · Messy/rotated-digit accuracy: 90.6%

What is this fingerprint for? ▾

Test accuracy 98.5% Input size 8×8...

Jump to a mode

Browse Sample Digits

Draw Your Own

Upload a Photo

Currently open: Draw Your Own

How does this actually work?

Why predictions can miss

Tips for the best results

About the training data

Confidence key

High confidence (≥ 80%)

CONVOLUTIONAL NEURAL NETWORK · IMPLEMENTED FROM SCRATCH

Handwritten Digit Recognition

A small CNN, implemented in raw NumPy with no deep learning framework, classifying handwritten digits in real time. Browse labeled training examples, draw your own digit, or upload a photo to see it in action.

Browse Sample Digits Draw Your Own Upload a Photo

- 1 Draw a single digit (0–9) in the black canvas below using your mouse, trackpad or touchscreen.
- 2 Make it thick and centered — use the brush size slider if it's too thin.
- 3 Press **Predict** when you're happy with it.

Brush size 22

Live model input preview

Auto-cropped, scaled and centered — this is what the model sees

Predict Clear canvas

Task 6 — Streamlit User Interface for Deep Learning Models · Model trained from scratch in NumPy in Task 1

Figure 5: A digit drawn with real mouse movements directly on the canvas, alongside the live 8x8 preprocessing preview - showing digit_utils.py's crop/scale/center pipeline working in real time as the stroke is drawn.

6.6 Draw Your Own - Prediction Result

Digit Classifier

A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.

Model info

Robust model loaded ✓

Fingerprint: 94e8348e89

Clean-digit accuracy: 98.5% · Messy/rotated-digit accuracy: 90.6%

What is this fingerprint for? ▾

Test accuracy 98.5% Input size 8×8...

Jump to a mode

Browse Sample Digits

Draw Your Own

Upload a Photo

Currently open: Draw Your Own

How does this actually work?

Why predictions can miss

Tips for the best results

About the training data

Confidence key

- High confidence (≥ 80%)
- Moderate confidence (50–80%)
- Low confidence (< 50%)

Reset app

CONVOLUTIONAL NEURAL NETWORK · IMPLEMENTED FROM SCRATCH

Handwritten Digit Recognition

A small CNN, implemented in raw NumPy with no deep learning framework, classifying handwritten digits in real time. Browse labeled training examples, draw your own digit, or upload a photo to see it in action.

Browse Sample Digits Draw Your Own Upload a Photo

- 1 Draw a single digit (0–9) in the black canvas below using your mouse, trackpad or touchscreen.
- 2 Make it thick and centered — use the brush size slider if it's too thin.
- 3 Press **Predict** when you're happy with it.

Brush size 22

Live model input preview

Auto-cropped, scaled and centered — this is what the model sees

Predict Clear canvas

Result

What the model actually saw (8×8 pixels)

PREDICTED DIGIT

2

54.5% · Moderate confidence

Top 3 guesses

- 2 — 54.5%
- 5 — 14.5%
- 0 — 9.9%

See the full probability breakdown (all 10 digits)

Task 6 — Streamlit User Interface for Deep Learning Models · Model trained from scratch in NumPy in Task 1

Figure 6: The resulting prediction - in this run the hand-drawn stroke was read as a 2 at 54.5% (Moderate) confidence rather than the intended 7. Shown here honestly rather than only showcasing successful predictions - a fairly minimal two-segment stroke is genuinely ambiguous at 8x8 resolution.

6.7 Upload a Photo - Empty State

<<

Digit Classifier

A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.

Model info

Robust model loaded ✓

Fingerprint: 94e8348e89

Clean-digit accuracy: 98.5% · Messy/rotated-digit accuracy: 90.6%

What is this fingerprint for? ▾

Test accuracy 98.5% · Input size 8×8...

Jump to a mode

Browse Sample Digits

Draw Your Own

Upload a Photo

Currently open: Upload a Photo

How does this actually work?

Why predictions can miss

Tips for the best results

About the training data

Confidence key

High confidence (≥80%)

Moderate confidence (50–80%)

Low confidence (< 50%)

Reset app

Deploy ⋮

CONVOLUTIONAL NEURAL NETWORK · IMPLEMENTED FROM SCRATCH

Handwritten Digit Recognition

A small CNN, implemented in raw NumPy with no deep learning framework, classifying handwritten digits in real time. Browse labeled training examples, draw your own digit, or upload a photo to see it in action.

Browse Sample Digits

Draw Your Own

Upload a Photo

1

Upload a clear photo or scan with a single handwritten digit.

2

Check the processed preview below — toggle **Invert colors** if it looks wrong (white background vs. black).

3

Press **Predict**.

Choose an image

Upload

200MB per file · PNG, JPG

Options

☒ Invert colors ⓘ

Best results: good lighting, digit fills most of the frame, plain background.

Upload a PNG or JPG photo of a handwritten digit to get started.

Task 6 — Streamlit User Interface for Deep Learning Models · Model trained from scratch in NumPy in Task 1

Figure 7: The photo-upload tab, with the invert-colors toggle and guidance on getting good results.

6.8 Upload a Photo - Three-Way Preview

Digit Classifier

A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.

Model info

Robust model loaded ✓

Fingerprint: 94e8348e89

Clean-digit accuracy: 98.5% · Messy/rotated-digit accuracy: 90.6%

What is this fingerprint for? ▾

Test accuracy 98.5% Input size 8×8...

Jump to a mode

Browse Sample Digits

Draw Your Own

Upload a Photo

Currently open: Upload a Photo

How does this actually work?

Why predictions can miss

Tips for the best results

About the training data

Confidence key

High confidence (≥ 80%)

Moderate confidence (50–80%)

Low confidence (< 50%)

Reset app

CONVOLUTIONAL NEURAL NETWORK · IMPLEMENTED FROM SCRATCH

Handwritten Digit Recognition

A small CNN, implemented in raw NumPy with no deep learning framework, classifying handwritten digits in real time. Browse labeled training examples, draw your own digit, or upload a photo to see it in action.

Browse Sample Digits

Draw Your Own

Upload a Photo

1 Upload a clear photo or scan with a single handwritten digit.

2 Check the processed preview below — toggle **Invert colors** if it looks wrong (white background vs. black).

3 Press **Predict**.

Choose an image

test_digit_photo.png 1.3KB +

Options

☒ Invert colors ⓘ

Best results: good lighting, digit fills most of the frame, plain background.

Original upload

Auto-cropped to the digit

What the model will see



If this doesn't look like a recognizable digit, try the invert toggle or a clearer photo.

Predict

Task 6 — Streamlit User Interface for Deep Learning Models · Model trained from scratch in NumPy in Task 1

Figure 8: The 3-way preview after uploading a test image of a “3” - original upload, auto-cropped to the ink, and the final 8x8 array the model will actually see.

6.9 Upload a Photo - Prediction Result

Deploy

⋮

CONVOLUTIONAL NEURAL NETWORK · IMPLEMENTED FROM SCRATCH

Handwritten Digit Recognition

A small CNN, implemented in raw NumPy with no deep learning framework, classifying handwritten digits in real time. Browse labeled training examples, draw your own digit, or upload a photo to see it in action.

Browse Sample Digits

Draw Your Own

Upload a Photo

1 Upload a clear photo or scan with a single handwritten digit.

2 Check the processed preview below — toggle **Invert colors** if it looks wrong (white background vs. black).

3 Press **Predict**.

Choose an image

test_digit_photo.png

1.3KB

+

Options

☒ Invert colors

Best results: good lighting, digit fills most of the frame, plain background.

Original upload

Auto-cropped to the digit

What the model will see

3

3

If this doesn't look like a recognizable digit, try the invert toggle or a clearer photo.

Predict

Result

What the model actually saw (8x8 pixels)

PREDICTED DIGIT

3

61.2% · Moderate confidence

Top 3 guesses

3 — 61.2%

2 — 12.4%

8 — 8.2%

See the full probability breakdown (all 10 digits)

Digit Classifier

A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.

Model info

Robust model loaded ✓

Fingerprint: 94e8348e89

Clean-digit accuracy: 98.5% · Messy/rotated-digit accuracy: 90.6%

What is this fingerprint for? ▾

Test accuracy

Input size

98.5%

8x8...

Jump to a mode

Browse Sample Digits

Draw Your Own

Upload a Photo

Currently open: Upload a Photo

How does this actually work?

Why predictions can miss

Tips for the best results

About the training data

Confidence key

High confidence (≥ 80%)

Moderate confidence (50–80%)

Low confidence (< 50%)

Reset app

Task 6 — Streamlit User Interface for Deep Learning Models - Model trained from scratch in NumPy in Task 1

Figure 9: A correct prediction from the uploaded photo - true digit 3, predicted 3 at 61.2% (Moderate) confidence.

6.10 Sidebar - Explanatory Info Panel

Digit Classifier

A convolutional neural network, built from scratch in NumPy — no TensorFlow or PyTorch involved.

Model info

Robust model loaded ✓

Fingerprint: 94e8348e99

Clean-digit accuracy: 98.5%
Messy/rotated-digit accuracy: 90.6%

What is this fingerprint for? ▾

Test accuracy 98.5%
Input size 8×8...

Jump to a mode

Browse Sample Digits

Draw Your Own

Upload a Photo

Currently open: Browse Sample Digits

How does this actually work?

Why predictions can miss

Before scoring anything, the app crops your input to just the ink, scales each axis to fill the frame without collapsing thin shapes, and centers it by mass — the same normalization used to build the training images. Loopy digits like an 8 or 6 specifically get extra care so they don't get crushed into a thin line that looks like a 1.

A few things can still trip the model up:

- It was trained on plain, single-stroke digits — ornate serifs or a crossed 7 (a bar through the stem) can look unlike anything it has seen
- Extremely thin or wobbly strokes lose detail once reduced to 8×8
- A digit drawn at a steep angle is harder than one drawn upright

Tips for the best results

About the training data

Confidence key

- High confidence (≥ 80%)
- Moderate confidence (50–80%)
- Low confidence (< 50%)

Reset app

CONVOLUTIONAL NEURAL NETWORK · IMPLEMENTED FROM SCRATCH

Handwritten Digit Recognition

A small CNN, implemented in raw NumPy with no deep learning framework, classifying handwritten digits in real time. Browse labeled training examples, draw your own digit, or upload a photo to see it in action.

Browse Sample Digits

Draw Your Own

Upload a Photo

1 Filter by digit (optional), then click Use this under any thumbnail below.

2 Press Predict to see what the model thinks.

Filter gallery by true digit ▾ Shuffle gallery

True label: 2

Use this

True label: 8

Use this

True label: 2

Use this

True label: 6

Use this

True label: 6

Use this

True label: 7

Use this

True label: 1

Use this

True label: 9

Use this

Selected sample — index 1001, true label 2

Predict

Result

What the model actually saw (8×8 pixels)

PREDICTED DIGIT

2

82.3% · High confidence

Top 3 guesses

2 — 82.3%

8 — 9.2%

0 — 6.2%

See the full probability breakdown (all 10 digits)

Probability

Digit

Task 6 — Streamlit User Interface for Deep Learning Models · Model trained from scratch in NumPy in Task 1

Figure 10: The sidebar's “Why predictions can miss” panel, expanded - documenting the exact preprocessing pipeline and known failure modes for the person using the app.

7. Results

Metric	Value
Gradient check (max relative error, analytic vs numerical)	1.10×10^{-7} (passed)
Training epochs	24
Augmented examples generated per original image	18
Clean-digit validation accuracy	98.52%
Messy/rotated-digit validation accuracy	90.59%
Model fingerprint (weights hash)	94e8348e89

8. Observations

- The numerical gradient check passed with a maximum relative error of 1.10×10^{-7} between the hand-derived analytic gradients and a finite-difference approximation - independently confirming the backpropagation implementation is mathematically correct before any training time is spent on it.
- Training on heavily augmented data (18 augmented variants per original image - rotation, independent-axis scaling, translation, elastic warp, stroke thickening/thinning, and noise) produces a model that generalizes far better to messy input than clean-only training would: 98.52% on clean digits and 90.59% on deliberately messy/rotated digits.
- The real drawing canvas (`streamlit-drawable-canvas`) worked correctly end-to-end in this environment; the app's fallback path for cases where that component fails to load is implemented but was not needed for these screenshots. The live model-input preview (Figure 5) confirms `digit_utils.py` 's crop-to-ink and center-by-mass logic runs correctly on a real, freehand-drawn stroke, not just on pre-processed dataset images.
- Both a correct prediction and a genuine misclassification are included in this report (Figures 2/3/9 vs. Figure 6) rather than only curated successes - a simple two-segment hand-drawn stroke intended as a "7" was read as a "2" at moderate (54.5%) confidence, which is consistent with the model's own documented limitation that "extremely thin or wobbly strokes lose detail once reduced to 8x8".
- The color-coded confidence system (green $\geq 80\%$, orange 50-80%, red $< 50\%$) correctly reflected the actual reliability of each prediction across all three input modes tested.
- The upload path's 3-way preview (original / auto-cropped / model-input) makes the otherwise invisible preprocessing step inspectable, which is exactly what let a genuine upload of a hand-drawn "3" be correctly classified at 61.2% confidence.

9. Known Limitations

- **Training set size and resolution** - with roughly 1,800 base training images at 8x8 resolution, this remains a small, simple model rather than a production-grade recognizer.
- **Extreme proportions** - a digit drawn far taller than it is wide (or vice versa) can become genuinely ambiguous once compressed to 8x8 - an information-loss limit of the format itself, not a bug in the code.
- **Stylistic mismatches** - the training data uses plain, single-stroke digits, so ornate serifs, a crossed "7", or heavily cursive strokes can look unlike anything the model has seen.

10. Conclusion

This interface closes the train/inference gap that limits naive digit-classifier deployments: the same crop-scale-center preprocessing runs on both sides, the model is trained on heavily augmented data that better resembles real freehand input, and its from-scratch backpropagation was independently verified with a numerical gradient check before any training took place. Testing against the live, running application - across all three input modes, using a real drawing canvas and a real uploaded photo rather than only bundled dataset images - confirmed the interface behaves correctly and reports its own confidence honestly, including on the one case where its prediction was wrong.